

A Guide to Understanding Convolutional Neural Networks (CNNs) using Visualization

SSaurabh

Last Updated : 22 Mar, 2020



Introduction

“How did your neural network produce this result?” This question has sent many data scientists into a tizzy. It’s easy to explain how a simple neural network works, but what happens when you increase the layers 1000x in a [computer vision](#) project?

Our clients or end users require interpretability – they want to know how our model got to the final result. We can’t take a pen and paper to explain how a deep neural network works. So how do we shed this “black box” image of neural networks?

By visualizing them! The clarity that comes with visualizing the different features of a neural network is unparalleled. This is especially true when we’re dealing with a convolutional neural network (CNN) trained on thousands and millions of images.



In this article, we will look at different techniques for visualizing convolutional neural networks. Additionally, we will also work on extracting insights from these visualizations for tuning our CNN model.

Note: This article assumes you have a basic understanding of Neural Networks and Convolutional Neural Networks. Below are three helpful articles to brush up or get started with this topic:

- [*A Comprehensive Tutorial to learn Convolutional Neural Networks from Scratch*](#)
- [*An Introductory Guide to Deep Learning and Neural Networks*](#)
- [*Fundamentals of Deep Learning – Starting with Artificial Neural Network*](#)

You can also learn CNNs in a step-by-step manner by enrolling in this free course: [Convolutional Neural Networks \(CNN\) from Scratch](#)

Table Of Contents

1. Why Should we use Visualization to Decode Neural Networks?
2. Setting up the Model Architecture

3. Accessing Individual Layers of a CNN
4. Filters – Visualizing the Building Blocks of CNNs
5. Activation Maximization – Visualizing what a Model Expects
6. Occlusion Maps – Visualizing what’s important in the Input
7. Saliency Maps – Visualizing the Contribution of Input Features
8. Class Activation Maps
9. Layerwise Output Visualization – Visualizing the Process

Why Should we use Visualization to Decode Neural Networks?

It’s a fair question. There are a number of ways to understand how a neural network works, so why turn to the off-beaten path of visualization?

Let’s answer this question through an example. Consider a project where we need to classify images of animals, like snow leopards and Arabian leopards. Intuitively, we can differentiate between these animals using the image background, right?

Both animals live in starkly contrasting habitats. The majority of the snow leopard images will have snow in the background while most of the Arabian leopard images will have a sprawling desert.



Here's the problem – the model will start classifying snow versus desert images. So, how do we make sure our model has correctly learned the distinguishing features between these two leopard types? The answer lies in the form of visualization.

“

Visualization helps us see what features are guiding the model's decision for classifying an image.

There are multiple ways to visualize a model, and we will try to implement some of them in this article.

Setting up the Model Architecture

I believe the best way of learning is by coding the concept. Hence, this is a very hands-on guide and I'm going to dive into the Python code straight away.

We will be using the VGG16 architecture with pretrained weights on the [ImageNet dataset](#) in this article. Let's first import the model into our program and understand its architecture.

We will visualize the model architecture using the '***model.summary()***' function in Keras. *This is a very important step before we get to the model building part.* We need to make sure the input and output shapes match our problem statement, hence we visualize the model summary.

```
1 #importing required modules
2 from keras.applications import VGG16
3 #loading the saved model
4 #we are using the complete architecture thus include_top=True
5 model = VGG16(weights='imagenet',include_top=True)
6 #show the summary of model
```


Below is the model summary generated by the above code:

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
predictions (Dense)	(None, 1000)	4097000
Total params: 138,357,544		
Trainable params: 138,357,544		
Non-trainable params: 0		

We have a detailed architecture of the model along with the number of trainable parameters at every layer. I want you to spend a few moments going through the above output to understand what we have at hand.

- 6 This is important when we are training only a subset of the model layers (feature extraction). We can generate the model summary and ensure that the number of non-trainable parameters matches the layers that we do not want to train.

Also, we can use the total number of trainable parameters to check whether our GPU will be able to allocate sufficient memory for training the model. That's a familiar challenge for most of us working on our personal machines!

Accessing Individual Layers

Now that we know how to get the overall architecture of a model, let's dive deeper and try to explore individual layers.

It's actually fairly easy to access the individual layers of a Keras model and extract the parameters associated with each layer. This includes the layer weights and other information like the number of filters.

Now, we will create dictionaries that map the layer name to its corresponding characteristics and layer weights:

```
1 #creating a mapping of layer name ot layer details
2 #we will create a dictionary layers_info which maps a layer name to its charcteristics
3 layers_info = {}
4 for i in model.layers:
5     layers_info[i.name] = i.get_config()
6
```



```

7  #here the layer_weights dictionary will map every layer_name to its corresponding weight
8  layer_weights = {}
9  for i in model.layers:
10     layer_weights[i.name] = i.get_weights()
11
12  print(layers_info['block5_conv1'])

```

Layers.py hosted with ❤ by GitHub

[view raw](#)

The above code gives the following output which consists of different parameters of the *block5_conv1* layer:

```

{'name': 'block5_conv1',
 'trainable': True,
 'filters': 512,
 'kernel_size': (3, 3),
 'strides': (1, 1),
 'padding': 'same',
 'data_format': 'channels_last',
 'dilation_rate': (1, 1),
 'activation': 'relu',
 'use_bias': True,
 'kernel_initializer': {'class_name': 'VarianceScaling',
 'config': {'scale': 1.0,
 'mode': 'fan_avg',
 'distribution': 'uniform',
 'seed': None}},
 'bias_initializer': {'class_name': 'Zeros', 'config': {}},
 'kernel_regularizer': None,
 'bias_regularizer': None,
 'activity_regularizer': None,
 'kernel_constraint': None,
 'bias_constraint': None}

```

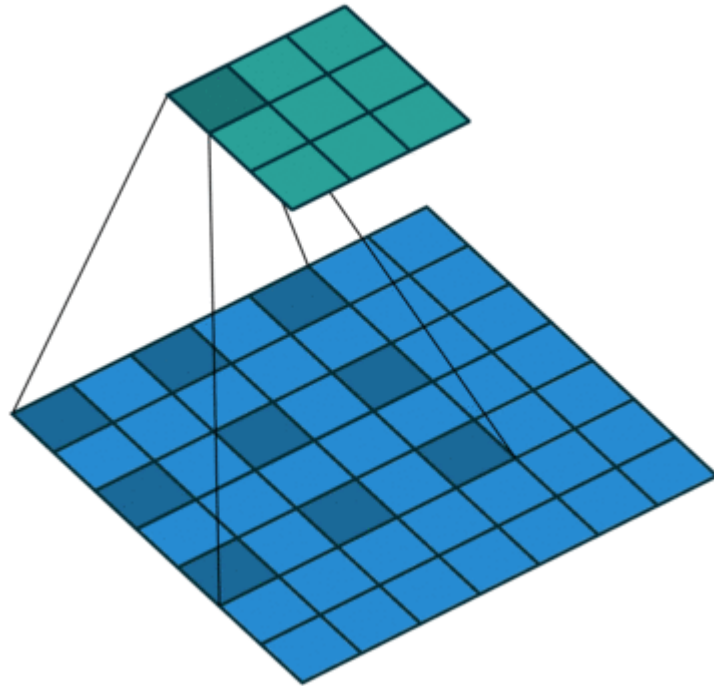
Did you notice that the trainable parameter for our layer '*block5_conv1*' is true? This means that we can update the layer weights by training the model further.

Visualizing the Building Blocks of CNNs – Filters

Filters are the basic building blocks of any [Convolutional Neural Network](#). Different filters extract different kinds of features from an image. The below GIF illustrates



this point really well:



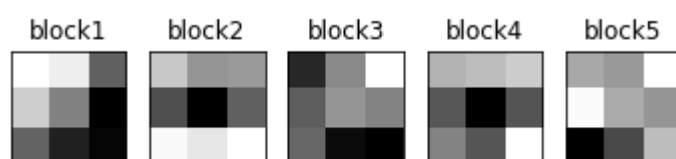
As you can see, every convolutional layer is composed of multiple filters. Check out the output we generated in the previous section – the ‘*block5_conv1*’ layer consists of 512 filters. Makes sense, right?

Let’s plot the first filter of the first convolutional layer of every VGG16 block:

```
1 layers = model.layers
2 layer_ids = [1,4,7,11,15]
3 #plot the filters
4 fig,ax = plt.subplots(nrows=1,ncols=5)
5 for i in range(5):
6     ax[i].imshow(layers[layer_ids[i]].get_weights()[0][:,:, :,0][:,:,0],cmap='gray')
7     ax[i].set_title('block'+str(i+1))
8     ax[i].set_xticks([])
9     ax[i].set_yticks([])
```

Filters.py hosted with ❤ by GitHub

[view raw](#)



We can see the filters of different layers in the above output. **All the filters are of the same shape since VGG16 uses only 3x3 filters.**

Visualizing what a Model Expects – Activation Maximization

Let's use the image below to understand the concept of activation maximization:



Which features do you feel will be important for the model to identify the elephant?

Some major ones I can think of:

- Tusks
- Trunk
- Ears

That's how we instinctively identify elephants, right? Now, let's see what we get when we try to optimize a random image to be classified as that of an elephant.

We know that every convolutional layer in a [CNN](#) looks for similar patterns in the output of the previous layer. The activation of a convolutional layer is maximized when the input consists of the pattern that it is looking for.

6

In the activation maximization technique, we update the input to each layer so that the activation maximization loss is minimized.

How do we do this? We calculate the gradient of the activation loss with respect to the input, and then update the input accordingly:

$$\frac{\partial \text{ActivationMaximizationLoss}}{\partial \text{input}}$$

Here's the code for doing this:

```
1  #importing the required modules
2  from vis.visualization import visualize_activation
3  from vis.utils import utils
4  from keras import activations
5  from keras import applications
6  import matplotlib.pyplot as plt
7  %matplotlib inline
8  plt.rcParams['figure.figsize'] = (18,6)
9  #creating a VGG16 model using fully connected layers also because then we can
10 #visualize the patterns for individual category
11 from keras.applications import VGG16
12 model = VGG16(weights='imagenet',include_top=True)
13
14 #finding out the layer index using layer name
15 #the find_layer_idx function accepts the model and name of layer as parameters and ret
16 layer_idx = utils.find_layer_idx(model,'predictions')
17 #changing the activation of the layer to linear
18 model.layers[layer_idx].activation = activations.linear
19 #applying modifications to the model
20 model = utils.apply_modifications(model)
21 #Indian elephant
22 img3 = visualize_activation(model,layer_idx,filter_indices=385,max_iter=5000,verbose=T
23 plt.imshow(img3)
```

activation_max.py hosted with ❤ by GitHub

[view raw](#)

Our model generated the below output using a random input for the class corresponding to Indian Elephant:





From the above image, we can observe that the model expects structures like a tusk, large eyes, and trunk. Now, this information is very important for us to check the sanity of our dataset. For example, let's say that the model was focussing on features like trees or long grass in the background because Indian elephants are generally found in such habitats.

Then, using activation maximization, we can figure out that our dataset is probably not sufficient for the task and we need to add images of elephants in different habitats to our training set.

Visualizing what's Important in the Input- Occlusion Maps

Activation maximization is used to visualize what the model expects in an image. Occlusion maps, on the other hand, help us find out which part of the image is important for the model.

Now, to understand how occlusion maps work, we consider a model that classifies cars according to their manufacturers, like Toyota, Audi etc.:





Can you figure out which company manufactured the above car? Probably not because the part where the company logo is placed has been occluded in the image. That part of the image is clearly important for our classification purposes.

Similarly, for generating an occlusion map, we occlude some part of the image and then calculate its probability of belonging to a class. If the probability decreases, then it means that occluded part of the image is important for the class. Otherwise, it is not important.

Here, we assign the probability as pixel values for every part of the image and then standardize them to generate a heatmap:

```
1  import numpy as np
2
3  from keras.utils import np_utils
4  from keras.models import Sequential
5  from keras.layers import Dense, Dropout, Flatten, Activation, Conv2D, MaxPooling2D
6  from keras.optimizers import Adam
7  from keras.callbacks import EarlyStopping, ModelCheckpoint
8  from keras.preprocessing.image import ImageDataGenerator
9  from keras.activations import relu
10
11 %matplotlib inline
12 import matplotlib.pyplot as plt
13 def iter_occlusion(image, size=8):
14
15     occlusion = np.full((size * 5, size * 5, 1), [0.5], np.float32)
```

```

16     occlusion_center = np.full((size, size, 1), [0.5], np.float32)
17     occlusion_padding = size * 2
18
19     # print('padding...')
20     image_padded = np.pad(image, ( \
21         (occlusion_padding, occlusion_padding), (occlusion_padding, oc
22         ), 'constant', constant_values = 0.0)
23
24     for y in range(occlusion_padding, image.shape[0] + occlusion_padding, size):
25
26         for x in range(occlusion_padding, image.shape[1] + occlusion_padding, size):
27             tmp = image_padded.copy()
28
29             tmp[y - occlusion_padding:y + occlusion_center.shape[0] + occlusion_paddin
30             x - occlusion_padding:x + occlusion_center.shape[1] + occlusion_paddin
31             = occlusion
32
33             tmp[y:y + occlusion_center.shape[0], x:x + occlusion_center.shape[1]] = oc
34
35         yield x - occlusion_padding, y - occlusion_padding, \
36             tmp[occlusion_padding:tmp.shape[0] - occlusion_padding, occlusion_pa

```

occlusion_1.py hosted with ❤ by GitHub

[view raw](#)

The above code defines a function *iter_occlusion* that returns an image with different masked portions.

Now, let's import the image and plot it:

```

1  from keras.preprocessing.image import load_img
2  # load an image from file
3  image = load_img('car.jpeg', target_size=(224, 224))
4  plt.imshow(image)
5  plt.title('ORIGINAL IMAGE')

```

occlusion_2.py hosted with ❤ by GitHub

[view raw](#)



Original Image

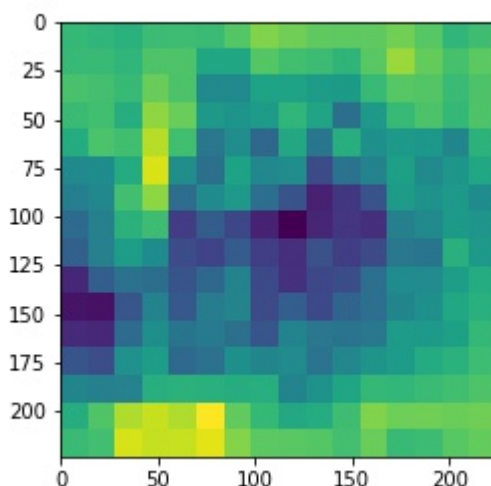
Now, we'll follow three steps:

- Preprocess this image
- Calculate the probabilities for different masked portions
- Plot the heatmap

```
1  from keras.preprocessing.image import img_to_array
2  from keras.applications.vgg16 import preprocess_input
3  # convert the image pixels to a numpy array
4  image = img_to_array(image)
5  # reshape data for the model
6  image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
7  # prepare the image for the VGG model
8  image = preprocess_input(image)
9  # predict the probability across all output classes
10 yhat = model.predict(image)
11 temp = image[0]
12 print(temp.shape)
13 heatmap = np.zeros((224,224))
14 correct_class = np.argmax(yhat)
15 for n,(x,y,image) in enumerate(iter_occlusion(temp,14)):
16     heatmap[x:x+14,y:y+14] = model.predict(image.reshape((1, image.shape[0], image.sha
17     print(x,y,n,' - ',image.shape)
18 heatmap1 = heatmap/heatmap.max()
19 plt.imshow(heatmap)
```

occlusion_3.py hosted with ❤ by GitHub

[view raw](#)



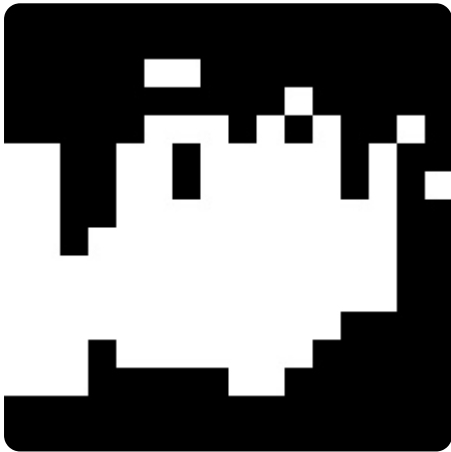
Heatmap

Really interesting. We will now create a mask using the standardized heatmap probabilities and plot it:

```
1 import skimage.io as io
2 #creating mask from the standardised heatmap probabilities
3 mask = heatmap1 < 0.85
4 mask1 = mask *256
5 mask = mask.astype(int)
6 io.imshow(mask, cmap='gray')
```

occlusion_4.py hosted with ❤ by GitHub

[view raw](#)



Mask

Finally, we will impose the mask on our input image and plot that as well:

```
1 import cv2
2 #read the image
3 image = cv2.imread('car.jpeg')
4 image = cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
5 #resize image to appropriate dimensions
6 image = cv2.resize(image,(224,224))
7 mask = mask.astype('uint8')
8 #apply the mask to the image
9 final = cv2.bitwise_and(image,image,mask = mask)
10 final = cv2.cvtColor(final,cv2.COLOR_BGR2RGB)
11 #plot the final image
12 plt.imshow(final)
```

occlusion_5.py hosted with ❤ by GitHub





Masked Car

Can you guess why we're seeing only certain parts? That's right – only those parts of the input image that had a significant contribution to its output class probability are visible. That, in a nutshell, is what occlusion maps are all about.

Visualizing the Contribution of Input Features- Saliency Maps

Saliency maps are another visualization technique based on gradients. These maps were introduced in the paper – [Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps.](#)

Saliency maps calculate the effect of every pixel on the output of the model. This involves calculating the gradient of the output with respect to every pixel of the input image.

This tells us how the output category changes with respect to small changes in the input image pixels. All the positive values of gradients mean that small changes to the pixel value will increase the output value:

$$\frac{\partial \text{output}}{\partial \text{input}}$$

These gradients, which are of the same shape as the image (gradient is calculated with respect to every pixel), provide us with the intuition of attention.

Let's see how to generate saliency maps for any image. First, we will read the input image using the below code segment.



Input Image

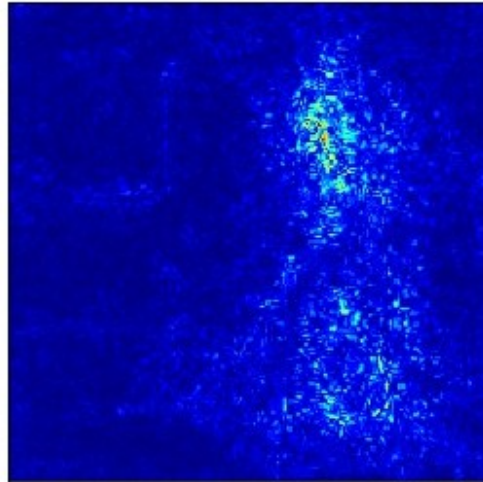
Now, we will generate the saliency map for the image using the VGG16 model:

```
1  # Utility to search for layer index by name.
2  # Alternatively we can specify this as -1 since it corresponds to the last layer.
3  layer_idx = utils.find_layer_idx(model, 'predictions')
4
5  # Swap softmax with linear
6  model.layers[layer_idx].activation = activations.linear
7  model = utils.apply_modifications(model)
8
9  #generating saliency map with unguided backprop
10 grads1 = visualize_saliency(model, layer_idx, filter_indices=None, seed_input=image)
11 #plotting the unguided saliency map
12 plt.imshow(grads1, cmap='jet')
```

saliency_2.py hosted with ❤ by GitHub

[view raw](#)

UNGUIDED



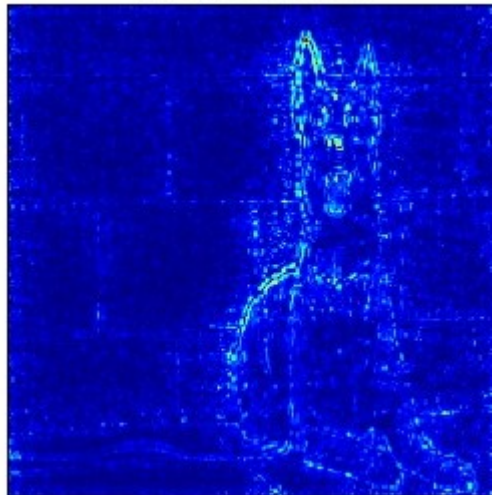
We see that the model focuses more on the facial part of the dog. Now, let's look at the results with guided backpropagation:

```
1 #generating saliency map with guided backprop
2 grads2 = visualize_saliency(model, layer_idx, filter_indices=None, seed_input=image, back
3 #plotting the saliency map as heatmap
4 plt.imshow(grads2, cmap='jet')
```

saliency_3.py hosted with ❤ by GitHub

[view raw](#)

GUIDED GRADIENTS



Guided backpropagation truncates all the negative gradients to 0, which means that only the pixels which have a positive influence on the class probability are updated.

Class Activation Maps (Gradient Weighted)

Class activation maps are also a neural network visualization technique based on the idea of weighing the activation maps according to their gradients or their contribution to the output.

The following excerpt from the Grad-CAM paper gives the gist of the technique:

- 6 Gradient-weighted Class Activation Mapping (Grad-CAM), uses the gradients of any target concept (say logits for ‘dog’ or even a caption), flowing into the final convolutional layer to produce a coarse localization map highlighting the important regions in the image for predicting the concept.

In essence, we take the feature map of the final convolutional layer and weigh (multiply) every filter with the gradient of the output with respect to the feature map.

Grad-CAM involves the following steps:

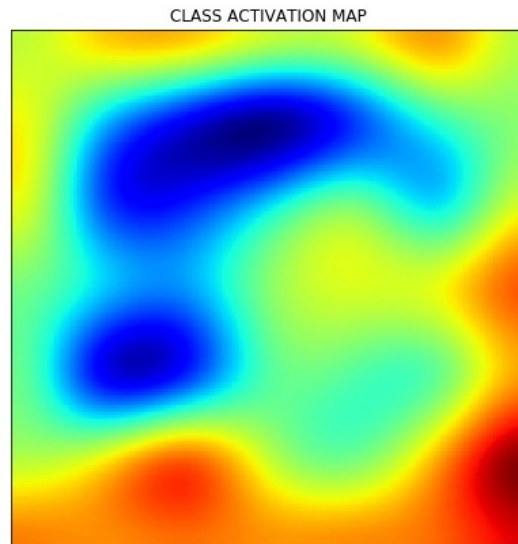
1. Take the output feature map of the final convolutional layer. The shape of this feature map is 14x14x512 for VGG16
2. Calculate the gradient of the output with respect to the feature maps
3. Apply Global Average Pooling to the gradients
4. Multiply the feature map with corresponding pooled gradients

We can see the input image and its corresponding Class Activation Map below:





Now let's generate the Class activation map for the above image.



Visualizing the Process – Layerwise Output Visualization

The starting layers of a [CNN](#) generally look for low-level features like edges. The features change as we go deeper into the model.

Visualizing the output at different layers of the model helps us see what features of the image are highlighted at the respective layer. This step is particularly important to fine-tune an architecture for our problems. Why? Because we can see which layers give what kind of features and then decide which layers we want to use in our model.

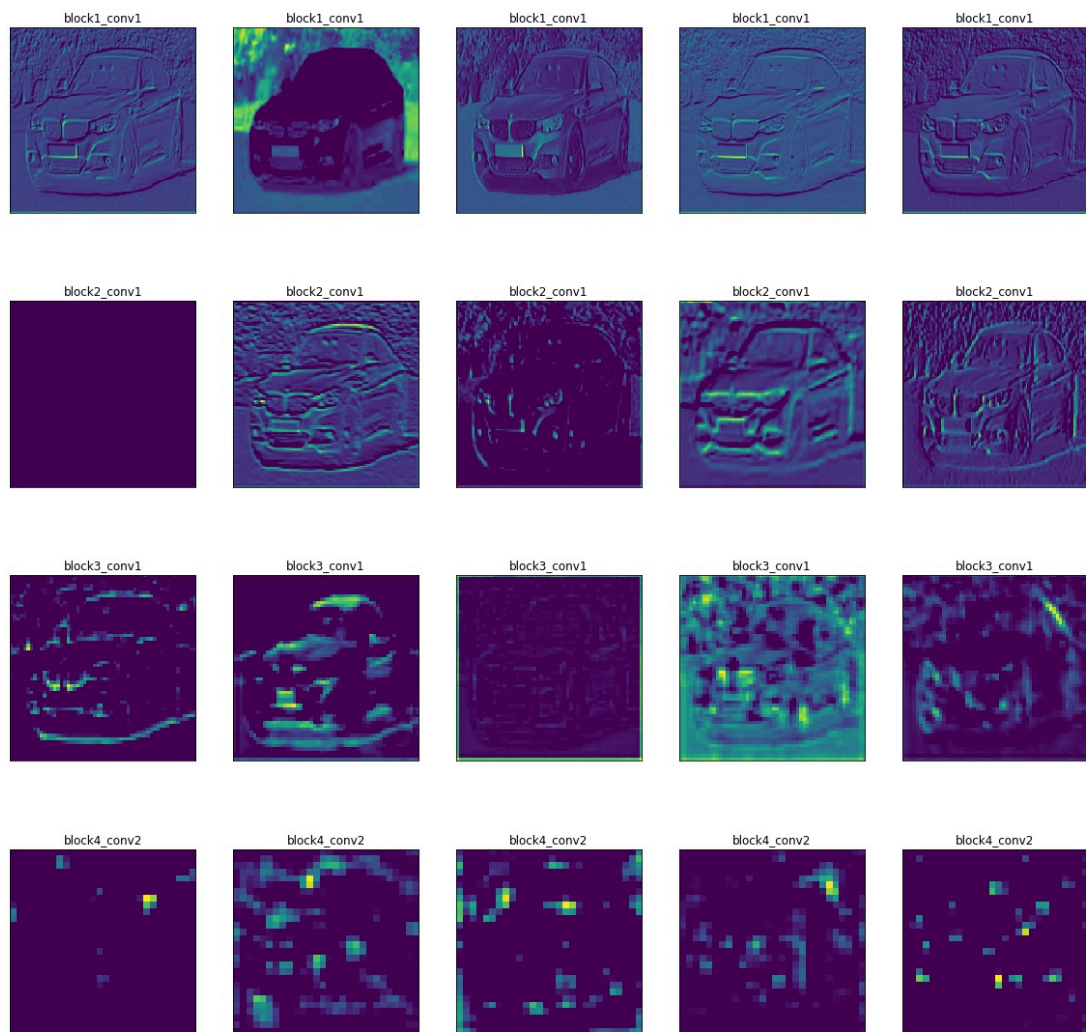
For example, visualizing layer outputs can help us compare the performance of different layers in the [neural style transfer](#) problem.

Let's see how we can get the output at different layers of a VGG16 model:

```
1  #importing required libraries and functions
2  from keras.models import Model
3  #defining names of layers from which we will take the output
4  layer_names = ['block1_conv1','block2_conv1','block3_conv1','block4_conv2']
5  outputs = []
6  image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
7  #extracting the output and appending to outputs
8  for layer_name in layer_names:
9      intermediate_layer_model = Model(inputs=model.input,outputs=model.get_layer(layer_
10      intermediate_output = intermediate_layer_model.predict(image)
11      outputs.append(intermediate_output)
12  #plotting the outputs
13  fig,ax = plt.subplots(nrows=4,ncols=5,figsize=(20,20))
14
15  for i in range(4):
16      for z in range(5):
17          ax[i][z].imshow(outputs[i][0,:,:,:z])
18          ax[i][z].set_title(layer_names[i])
19          ax[i][z].set_xticks([])
20          ax[i][z].set_yticks([])
21  plt.savefig('layerwise_output.jpg')
```

layerwise_output.py hosted with ❤ by GitHub

[view raw](#)



The above image shows the different features that are extracted from the image by every layer of VGG16 (except block 5). We can see that the starting layers correspond to low-level features like edges, whereas the later layers look at features like the roof, exhaust, etc. of the car.

End Notes

Visualization never ceases to amaze me. There are multiple ways to understand how a technique works, but visualizing it makes it a whole lot more fun. Here are a couple of resources you should check out:

- The process of feature extraction in neural networks is an active research area and has led to the development of awesome tools like Tensorspace and [Activation Atlases](#)
- [TensorSpace](#) is also a neural network visualization tool that supports multiple model formats. It lets you load your model and visualize it interactively. TensorSpace also has a playground where multiple architectures are available for visualization which you can play around with

Let me know if you have any questions or feedback on this article. I'll be happy to get into a discussion!

S [Saurabh](#)

A Data Science enthusiast and Software Engineer by training, Saurabh aims to work at the intersection of both fields.

Advanced

Algorithm

Computer Vision

Deep Learning

Image

Python

Python

Unstructured Data

Free Courses

